VVSS, Lab 4: Testing Levels

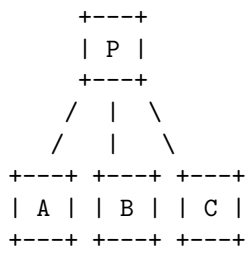
Computer Science – 2025-2026/II

Objective

Perform **integration testing** using two strategies: **Big-Bang** and **Top-Down**. You will write unit tests and integration tests using JUnit and Spring Boot.

Background

We assume the application has the following module dependency diagram:



- **P** – the complete application
- **A** – module for functionality **F01**
- **B** – module for functionality **F02**
- **C** – module for functionality **F03**

Look at the given Spring Boot application and **choose 3 modules** from the codebase that will serve as A, B, and C. Read the **README** file for an overview of the application and its modules.

> **How to choose modules A, B, C:**
>
> Modules must be chosen based on **functional communication**, not data-model dependency. Ask yourself: does module A *call* module B as part of its functionality? Not: does entity A *reference* entity B in its fields?
>
> **Bad example:** User → Recipe → Review. These entities reference each other, but that is a data-model relationship, not a functional one between components.
>
> **Good example:** User → Recipe → Ranking. `RankingService` internally calls both `ReviewService` and `CommentService` to compute rankings – this is a real functional dependency between components, which is what integration tests should exercise.

Once you have chosen your three modules, find a **flow in which all three participate** and use that flow as the basis for your Big-Bang test. The same three modules are then used in your incremental tests.

If you have trouble finding suitable modules, you may extend the project with additional modules and functionalities – but you **must** contact the instructor first.

What to Submit

You will create the following **7 test files** (stubs are already provided in `src/test/java/.../example_for_homework/`):

Unit tests:

- `ModuleAUnitTests.java` – Unit test for module A in isolation
- `ModuleBUnitTests.java` – Unit test for module B in isolation
- `ModuleCUnitTests.java` – Unit test for module C in isolation

Big-Bang integration:

- `BigBangIntegrationTest.java` – Integration test combining all modules at once

Top-Down incremental integration:

- `IncrementalIntegrationModuleATest.java` – Integrate A only (mock B and C)
- `IncrementalIntegrationModuleBTest.java` – Integrate A + B (mock C)
- `IncrementalIntegrationModuleCTest.java` – Integrate A + B + C (no mocks)

Example tests are provided in the `services/`, `controllers/`, and `repositories/` test packages. A mocking example is provided in `example_for_homework/MockingExampleTest.java`.

Task 1: Unit Tests

Write **one unit test per module** (A, B, C) in the corresponding stub files. Each test should verify the module's functionality **in isolation**, using mocks for any dependencies.

See `services/RecipeServiceUnitTest.java` for an example of a unit test with Mockito.

> **Note:** Do **not** write unit tests for Spring Data repositories (e.g., `UserRepository`, `RecipeRepository`). Spring already tests its own repository layer – there is nothing for you to verify there. Only add repository-level tests if you have written **custom query logic** of your own.

Task 2: Big-Bang Integration

In `BigBangIntegrationTest.java`, write **one integration test** that tests all three modules together in a single scenario.

What is Big-Bang integration?

All modules are integrated and tested simultaneously. No module is tested in combination with others beforehand – everything comes together at once. The test is a single method that executes a complete scenario spanning all three modules.

The scenario should mimic a **realistic user flow** through the application – a sequence of actions a real user would perform, where each step depends on the result of the previous one. This is what makes it a true integration test: the modules must actually talk to each other to carry the flow forward.

For example: - login → list recipes → add a comment to a recipe - login → add a new recipe → add a comment → submit a review

Choose one execution path, e.g.: P -> A -> B -> C.

Task 3: Top-Down Incremental Integration

Write **3 integration tests**, one per file, that progressively add modules:

Step 1 – Integrate A (`IncrementalIntegrationModuleATest.java`): - Test P with only module A. - Use `@MockBean` to mock the services of B and C. - Path: P -> A.

Step 2 – Integrate B (`IncrementalIntegrationModuleBTest.java`): - Test P with modules A and B together. - Use `@MockBean` to mock the service of C. - Pick one path: P -> A -> B **or** P -> B -> A.

Step 3 – Integrate C (`IncrementalIntegrationModuleCTest.java`): - Test P with all modules (A, B, and C). No mocks needed. - Pick one path, e.g.: P -> A -> B -> C, P -> B -> A -> C, P -> A -> C -> B, etc.

What is Top-Down integration?

Modules are integrated one at a time, starting from the top. Modules that haven't been integrated yet are replaced by mocks (using `@MockBean`). At each

step, you add one real module and verify that it **correctly communicates** with the previously integrated ones.

What to verify at each step:

At each incremental step, you are checking that the newly added module integrates correctly with the one above it. There are two valid ways to do this:

1. **Verify the call itself** – use a spy or mock to assert that the expected method was actually called with the correct arguments (e.g., assert that `UserController` called `UserService.getById` with the right id).
2. **Verify the result of the call** – assert on the observable side-effect (e.g., something was saved to the database, or the correct value was returned).

   Note: Saving a `User` and a `Review` to the database and then checking that both records exist is a **persistence-layer** integration test – it verifies that the service layer talks to the database correctly. It is *not* the kind of cross-module integration test required here, which should verify that one functional module correctly uses another (e.g., `UserController` correctly calls `UserService.getById`).

See `example_for_homework/MockingExampleTest.java` for an example of how to use `@MockBean` to replace a service with a mock in a Spring Boot test.

Summary

Deliverable	Count
Unit tests (one per module)	3
Big-Bang integration test	1
Top-Down integration tests	3
Total test files	**7**